

Auto-Director AI: Cinematic Micro-Drama Architect

Comprehensive Developer Interview Guide

1. Project Abstract

Auto-Director is a highly resilient, multi-agent AI pipeline designed to instantly translate raw, misspelled, multilingual (Hinglish/English) text scripts into fully animated, voice-narrated, and subtitle-aligned 9:16 vertical micro-dramas optimized for YouTube Shorts and Instagram Reels.

The software bypasses traditional terminal execution by serving an end-to-end glowing cyberpunk-themed Streamlit User Interface, equipped securely for cloud deployment.

2. Core Technology Stack

- **Core Language:** Python 3.12+ (Object-Oriented, fully typed with Pydantic).
 - **Frontend / Cloud:** Streamlit, deployed to Streamlit Community Cloud (bypassing Vercel's severe serverless timeout barriers).
 - **Concurrency:** `concurrent.futures.ThreadPoolExecutor` for parallelizing heavy TTS and Video generation API calls.
 - **LLM Engine:** OpenRouter API (`meta-llama/llama-3.1-8b-instruct`) for scriptwriting, dialogue generation, and prompt translation.
 - **Visual Engine:** Replicate API (`MiniMax video-01`) for cinematic Text-to-Video. (Fallback: Pollinations AI & Stock Photos).
 - **Voice Synthesis:** ElevenLabs premium TTS. (Fallback: `gTTS` and `espeak-ng`).
 - **Video Assembly:** `MoviePy 2.x` and `FFmpeg` (H.264 muxing), manipulating native Python `Pillow` (PIL) for high-performance subtitle overlaying.
-

3. The 5-Agent Pipeline Architecture

This project is structurally divided into 5 autonomous, provider-agnostic agents that operate in sequence:

1. **Scriptwriter Agent:** Connects to OpenRouter. Receives raw unstructured user text and coerces the LLM to output a strict JSON list of "Scenes", containing exact Voice Narration lines, translated English visual prompts, and scene Mood classifications.
2. **Voice Agent:** Parses the JSON scenes in parallel to synthesize audio via ElevenLabs. Calculates exact waveform durations (`AudioFileClip.duration`) to dictate the pacing of the video. Enforces a strict 90-second overall truncation rule.
3. **Visual Agent:** Generates massive AI Video files on Replicate. Designed natively with a 3-layer error fallback cascade if API cloud servers crash.
4. **Subtitle Agent:** Consumes the narration strings and temporal durations. Computes exact, word-bounded timestamp alignments using proportional mapping.
5. **Director Agent:** Instantiates the `MoviePy 2.x` engine. Composites the H.264 video streams, burns in natively rendered `Pillow` 50pt stroke-width subtitles, attaches the

voiceover sequence, loops short cinematic clips dynamically, and stitches the entire timeline into a single render-ready .mp4 file.

4. Interview talking points: Problems Faced & Engineering Solutions

If an interviewer asks, *"Tell me about a difficult problem you ran into while building this and how you solved it?"*, here are your detailed architectural wins:

The "Free-Tier Limit" Crash & The Dynamic Extension Fallback

Problem: Video generation APIs are extremely resource-heavy. When testing the application, the Replicate cloud server strictly rejected the request with a 402 Free time limit reached error, which originally caused the entire pipeline to crash. **Solution:** I programmed a "3-Layer Safety Net" inside the Visual Agent. If the Replicate API fails, an `except` block catches the network timeout and specifically rewrites the target file's extension from `.mp4` back to `.png`. It then silently falls back to Pollinations AI (which is completely free) to fetch a beautiful cinematic static image. Our Director Agent is built to natively detect if an incoming file is a static `.png` instead of an `.mp4` video, and automatically injects a simulated "Ken Burns" pan-and-zoom movement effect into the still image so the final video doesn't feel frozen!

The Multi-Button UX Confusion & The Pre-Analysis Boundary

Problem: The 90-second YouTube restriction demands roughly ~130 words. In early versions, if a user submitted 500 words, the backend would simply cut off the video halfway through the story. We initially offered a confusing "Analyze Text" button separate from the "Generate" button. **Solution:** I rebuilt a seamless front-end validation layer. Now, there is only one "Generate" button. If the `word_count < 130`, the pipeline launches in the background immediately. If it exceeds 130, the UI detects it, pauses the generation, makes a swift side-call to OpenRouter, and presents two beautifully compressed, AI-generated alternative plots designed to fit exactly within the 90-second constraint. The user selects an option, and the pipeline continues normally.

Security Vulnerabilities in Cloud Deployments

Problem: After migrating the web UI to Streamlit Community Cloud (since Vercel failed due to 10-second timeout constraints), I realized that placing API Keys directly into `st.text_input` boxes caused the application to actively broadcast my server's private backend Keys into the frontend (masked uniquely as dots). **Solution:** I fundamentally changed the UI architecture layout. The frontend boxes were wiped strictly to `value=""` with a customized placeholder. The actual backend uses keys explicitly locked down inside Streamlit Cloud's `Secrets` vault. This completely secured my API quota while still allowing public users to optionally paste in their *own* personal API keys to safely override the server!

The MoviePy 2.x Breaking Upgrade

Problem: During final video assembly, the system violently aborted with `ImportError: cannot import name 'loop' from 'moviepy.video.fx'`. **Solution:** I discovered the Python environment installed the newly released MoviePy 2.2.1. The old `clip.fx(loop)` syntax was entirely deprecated. After executing an underlying Python

structure check on the library tree, I migrated our assembly scripts to conform to the new Effect design pattern, passing `clip.with_effects([vfx.Loop(duration)])` natively through the rendering pipeline.

5. Potential Scalability (What's Next?)

To take this to enterprise production, the next steps would be:

- Migrating from synchronous `Requests` over to `AIOHTTP` and `WebSockets` to provide the front-end user with live streamable video-stitching metrics in real-time.
- Developing exact Lip-Sync engine integrations (e.g. `SadTalker` / `Wav2Lip`) to map the generated TTS audio dynamically against generated AI face meshes.